1.导入模块

```
import itertools
# 导入相关模块
import warnings
import matplotlib.pyplot as plt
import seaborn as sns
from mlxtend.frequent_patterns import apriori
from mlxtend.frequent_patterns import association_rules

import pandas as pd

warnings.filterwarnings('ignore')
```

2.导入数据集

```
# 导入数据集并删除 V1-V5 列
movies_data = pd.read_csv('my_movies.csv')
movies_data = movies_data.drop(['V1', 'V2', 'V3', 'V4', 'V5'], axis=1)

# 查看数据前几行
movies_data.head()
```

```
   Sixth Sense  Gladiator  LOTR1  Harry Potter1  Patriot  LOTR2  \
0            1          0      1              1        0      1
1            0          1      0              0        1      0
2            0          0      1              0        0      1
3            1          1      0              0        1      0
4            1          1      0              0        1      0

   Harry Potter2  LOTR  Braveheart  Green Mile
0              0     0           0           1
1              0     0           1           0
2              0     0           0           0
3              0     0           0           0
4              0     0           0           0
```

3.数据理解

```
# 查看数据维度
movies_data.shape
```

```
(10, 10)
```

```
# 检查数据每列缺失值
movies_data.isna().sum()
```

```
Sixth Sense      0
Gladiator        0
```

```
LOTR1             0
Harry Potter1     0
Patriot           0
LOTR2             0
Harry Potter2     0
LOTR              0
Braveheart        0
Green Mile        0
dtype: int64
```

```
# 查看数据整体结构
movies_data.info(memory_usage='deep')
```

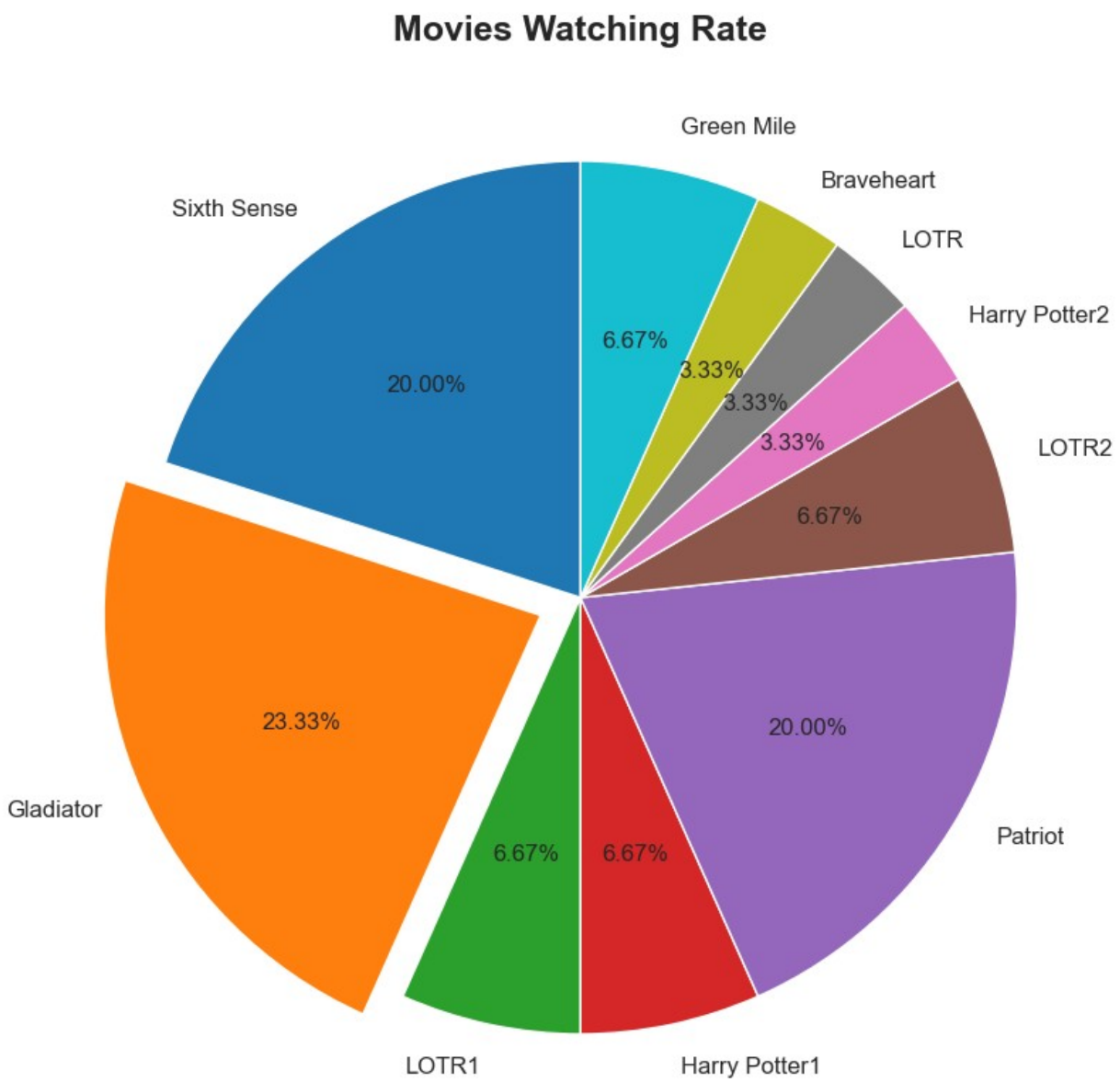
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 10 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Sixth Sense    10 non-null     int64
 1   Gladiator      10 non-null     int64
 2   LOTR1          10 non-null     int64
 3   Harry Potter1  10 non-null     int64
 4   Patriot        10 non-null     int64
 5   LOTR2          10 non-null     int64
 6   Harry Potter2  10 non-null     int64
 7   LOTR           10 non-null     int64
 8   Braveheart     10 non-null     int64
 9   Green Mile     10 non-null     int64
dtypes: int64(10)
memory usage: 932.0 bytes
```

```
# 统计每部电影的出现频率
count = movies_data.sum().sort_values(ascending=False)
count
```

```
Gladiator         7
Sixth Sense       6
Patriot           6
LOTR1             2
Harry Potter1     2
LOTR2             2
Green Mile        2
Harry Potter2     1
LOTR              1
Braveheart        1
dtype: int64
```

4.EDA 分析

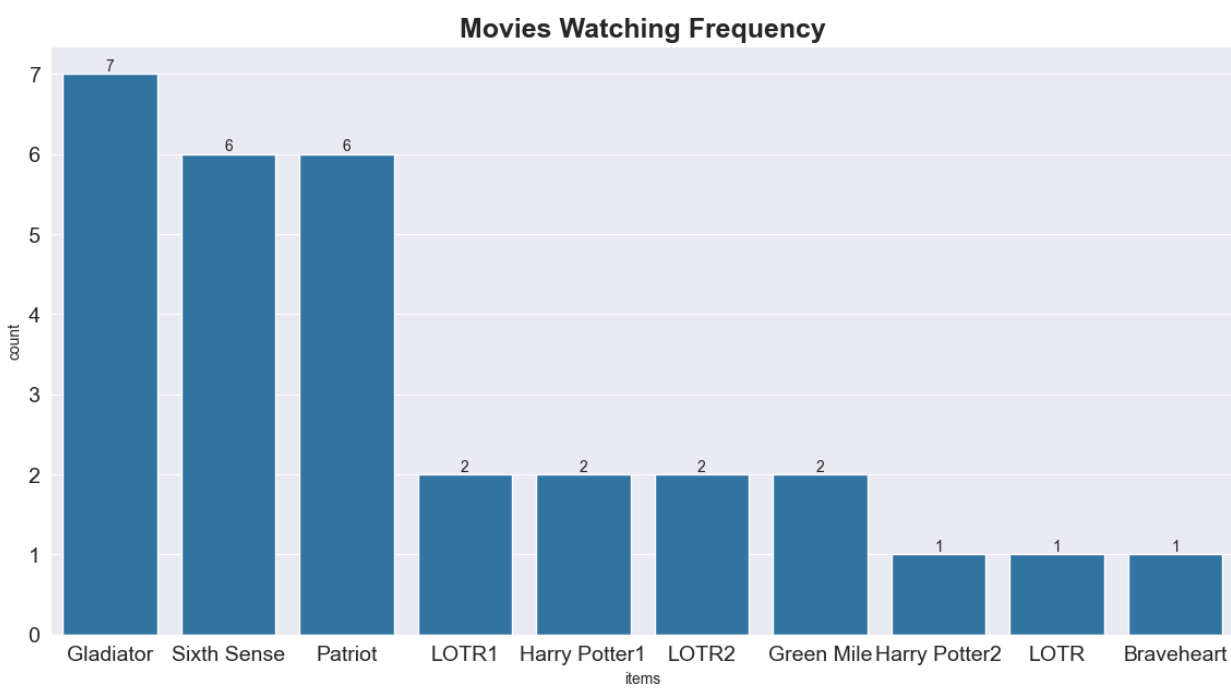
```
# 饼图
plt.figure(figsize=(10, 10))
plt.pie(movies_data.sum(), labels=movies_data.columns,
        explode=[0.0, 0.1, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0],
        autopct='%.2f%%', textprops={'size': 12}, startangle=90)

plt.title("Movies Watching Rate", fontsize=18, fontweight='bold')
plt.show()
```



```python
# 统计每部电影出现次数
count = count.to_frame().reset_index()
count = count.rename(columns={'index': 'items', 0: 'count'})
count
```

```
           items  count
0      Gladiator      7
1    Sixth Sense      6
2        Patriot      6
3          LOTR1      2
4  Harry Potter1      2
5          LOTR2      2
6     Green Mile      2
7  Harry Potter2      1
8           LOTR      1
9     Braveheart      1
```

```python
# 绘制条形图展示每部电影的频率
plt.figure(figsize=(14, 7))
ax = sns.barplot(x='items', y='count', data=count)
plt.xticks(fontsize=14)
plt.yticks(fontsize=14)
plt.title('Movies Watching Frequency', fontsize=18, fontweight='bold')
for i in ax.containers:
    ax.bar_label(i, )
```



5.N 项集及其支持度

```
# 计算1-项集的支持度
first = pd.DataFrame(columns=['Support'], data=movies_data.sum() /
movies_data.shape[0])
first
```

```
                Support
Sixth Sense         0.6
Gladiator           0.7
LOTR1               0.2
Harry Potter1       0.2
Patriot             0.6
LOTR2               0.2
Harry Potter2       0.1
LOTR                0.1
Braveheart          0.1
Green Mile          0.2
```

```
# 统计1-项集中支持度大于0.3的项集
first[first.Support >= 0.30]
```

```
              Support
Sixth Sense       0.6
Gladiator         0.7
Patriot           0.6
```

```
# 构造2-项集
second = list(itertools.combinations(movies_data.columns, 2))
second = [list(i) for i in second]
second[:5]
```

```
[['Sixth Sense', 'Gladiator'],
 ['Sixth Sense', 'LOTR1'],
 ['Sixth Sense', 'Harry Potter1'],
 ['Sixth Sense', 'Patriot'],
 ['Sixth Sense', 'LOTR2']]
```

```
# 计算2-项集的支持度，统计2-项集中支持度大于0.3的项集
value = []
for i in range(0, len(second)):
    temp = movies_data.T.loc[second[i]].sum()
    temp = len(temp[temp == movies_data.T.loc[second[i]].shape[0]]) /
movies_data.shape[0]
    value.append(temp)

secondIteration = pd.DataFrame({'Support': value})
secondIteration['index'] = [tuple(i) for i in second]
secondIteration['length'] = secondIteration['index'].apply(lambda x:
len(x))
secondIteration = secondIteration.sort_values(by='Support',
```

```
ascending=False).set_index('index')
secondIteration = secondIteration[secondIteration.Support >= 0.3]
secondIteration
```

```
                            Support  length
index
(Gladiator, Patriot)          0.6       2
(Sixth Sense, Gladiator)      0.5       2
(Sixth Sense, Patriot)        0.4       2
```

```
# 构造3-项集
second = list(itertools.combinations(movies_data.columns, 3))
second = [list(i) for i in second]
second[:5]
```

```
[['Sixth Sense', 'Gladiator', 'LOTR1'],
 ['Sixth Sense', 'Gladiator', 'Harry Potter1'],
 ['Sixth Sense', 'Gladiator', 'Patriot'],
 ['Sixth Sense', 'Gladiator', 'LOTR2'],
 ['Sixth Sense', 'Gladiator', 'Harry Potter2']]
```

```
# 计算3-项集的支持度，统计3-项集中支持度大于0.3的项集
value = []
for i in range(0, len(second)):
    temp = movies_data.T.loc[second[i]].sum()
    temp = len(temp[temp == movies_data.T.loc[second[i]].shape[0]]) /
movies_data.shape[0]
    value.append(temp)

secondIteration = pd.DataFrame({'Support': value})
secondIteration['index'] = [tuple(i) for i in second]
secondIteration['length'] = secondIteration['index'].apply(lambda x:
len(x))
secondIteration = secondIteration.sort_values(by='Support',
ascending=False).set_index('index')
secondIteration = secondIteration[secondIteration.Support >= 0.3]
secondIteration
```

```
                                    Support  length
index
(Sixth Sense, Gladiator, Patriot)      0.4       3
```

```
# 构造4-项集
second = list(itertools.combinations(movies_data.columns, 4))
second = [list(i) for i in second]
second[:5]
```

```
[['Sixth Sense', 'Gladiator', 'LOTR1', 'Harry Potter1'],
 ['Sixth Sense', 'Gladiator', 'LOTR1', 'Patriot'],
 ['Sixth Sense', 'Gladiator', 'LOTR1', 'LOTR2'],
```

```
 ['Sixth Sense', 'Gladiator', 'LOTR1', 'Harry Potter2'],
 ['Sixth Sense', 'Gladiator', 'LOTR1', 'LOTR']]
```

```
# 计算4-项集的支持度，统计4-项集中支持度大于0.3的项集
value = []
for i in range(0, len(second)):
    temp = movies_data.T.loc[second[i]].sum()
    temp = len(temp[temp == movies_data.T.loc[second[i]].shape[0]]) /
movies_data.shape[0]
    value.append(temp)

secondIteration = pd.DataFrame({'Support': value})
secondIteration['index'] = [tuple(i) for i in second]
secondIteration['length'] = secondIteration['index'].apply(lambda x:
len(x))
secondIteration = secondIteration.sort_values(by='Support',
ascending=False).set_index('index')
secondIteration = secondIteration[secondIteration.Support >= 0.3]
secondIteration
```

```
Empty DataFrame
Columns: [Support, length]
Index: []
```

无频繁 4-项集

6.求频繁项集，挖掘关联规则

```
# 设置min_support = 0.3
frequent_itemsets = apriori(min_support=0.3, use_colnames=True,
df=movies_data)
frequent_itemsets['length'] =
frequent_itemsets['itemsets'].apply(lambda x: len(x))
frequent_itemsets
```

```
   support                            itemsets  length
0      0.6                       (Sixth Sense)       1
1      0.7                         (Gladiator)       1
2      0.6                           (Patriot)       1
3      0.5            (Gladiator, Sixth Sense)       2
4      0.4              (Patriot, Sixth Sense)       2
5      0.6                (Gladiator, Patriot)       2
6      0.4  (Gladiator, Patriot, Sixth Sense)       3
```

```
# 设置confidence 的min_threshold = 0.6
rules = association_rules(frequent_itemsets, metric='confidence',
min_threshold=0.6)
rules = rules.sort_values(['lift','confidence','support'],
ascending=False).reset_index(drop=True)
rules =
```

```
rules[['antecedents','consequents','support','confidence','lift']]

# 设置lift > 1
rules = rules[rules.lift>1]
rules
```

```
                  antecedents                consequents  support
confidence  \
0                (Gladiator)                  (Patriot)      0.6
0.857143
1                  (Patriot)                (Gladiator)      0.6
1.000000
2     (Patriot, Sixth Sense)                (Gladiator)      0.4
1.000000
3   (Gladiator, Sixth Sense)                  (Patriot)      0.4
0.800000
4                  (Patriot)  (Gladiator, Sixth Sense)      0.4
0.666667
5              (Sixth Sense)                (Gladiator)      0.5
0.833333
6                (Gladiator)              (Sixth Sense)      0.5
0.714286
7                  (Patriot)              (Sixth Sense)      0.4
0.666667
8              (Sixth Sense)                  (Patriot)      0.4
0.666667
9       (Gladiator, Patriot)              (Sixth Sense)      0.4
0.666667
10             (Sixth Sense)      (Gladiator, Patriot)      0.4
0.666667

        lift
0   1.428571
1   1.428571
2   1.428571
3   1.333333
4   1.333333
5   1.190476
6   1.190476
7   1.111111
8   1.111111
9   1.111111
10  1.111111
```

可以得到 11 条各部电影之间的强关联规则：

- {Gladiator} → {Patriot}
- {Patriot} → {Gladiator}
- {Patriot, Sixth Sense} → {Gladiator}
- {Sixth Sense, Gladiator} → {Patriot}

- {Patriot} → {Sixth Sense, Gladiator}
- {Sixth Sense} → {Gladiator}
- {Gladiator} → {Sixth Sense}
- {Patriot} → {Sixth Sense}
- {Sixth Sense} → {Patriot}
- {Patriot, Gladiator} → {Sixth Sense}
- {Sixth Sense} → {Patriot, Gladiator}